LAB06 - Subconsultas

Subconsultas

Objetivos

Aprender a utilizar subconsultas dentro de una sentencia SQL:

1. **Subconsultas en una sentencia SELECT**
   - **Subconsultas no correlacionadas**
     - La subconsulta devuelve un solo valor
     - La subconsulta devuelve un conjunto de filas
   - **Subconsultas correlacionadas**
     - La subconsulta devuelve un solo valor
     - La subconsulta devuelve un conjunto de filas
2. **Subconsultas en UPDATES y DELETES**

En esta sesión trabajaremos con la BD Empresa completa en SQL Snippets (Empresa BDE).

Ejercicio 01 - Repaso JOIN

La dirección de la empresa necesita un informe completo sobre la situación de cada departamento. Debes crear una consulta que muestre:

- **nombre_departamento**: Nombre del departamento
- **nombre_director**: Nombre completo del director (nombre y apellido1 separados por espacio)
- **num_empleados**: Número total de empleados del departamento (sin contar al director)
- **num_proyectos**: Número de proyectos asignados al departamento

Requisitos importantes:

1. Deben aparecer **TODOS los departamentos**, incluso si no tienen proyectos asignados o empleados trabajando
2. Ten cuidado con el uso de `COUNT()` cuando uses `LEFT JOIN`
3. Ordena el resultado por número de empleados (de mayor a menor)

Solución:

Tabla resultado:

nombre_departamento	nombre_director	num_empleados	num_proyectos
Investigación	Alberto Campos	3	3
Administración	Juana Sáinz	2	2
Sede Central	Eduardo Ochoa	0	1

Subconsultas

Una **subconsulta** es una **consulta dentro de otra consulta**, utilizada para generar **resultados intermedios que pueden ser referenciados en la consulta principal**. Es decir, se trata de una consulta "anidada" que se ejecuta primero para proporcionar datos a la consulta externa.



Algunas características de las subconsultas son:

- **Modularidad**: permiten dividir una consulta compleja en partes más manejables, ayudando a mejorar la legibilidad y organización del código.
- **Filtrado avanzado**: permiten filtrar resultados basados en información derivada de otras tablas o cálculos, algo que no siempre es posible con operadores simples.
- **Optimización de consultas**: pueden mejorar el rendimiento al trabajar con subconjuntos de datos más específicos.
- **Flexibilidad**: permiten combinar datos de maneras muy flexibles, al usarse en múltiples secciones de una consulta: en la cláusula `SELECT`, `WHERE`, o `FROM`.

Subconsultas No Correlacionadas

Subconsultas No Correlacionadas

Una **subconsulta no correlacionada** es un tipo de subconsulta en SQL que se ejecuta de manera independiente de la consulta principal. Esto significa que la subconsulta **se evalúa**

solo una vez y sus resultados se utilizan en la consulta externa.

Este tipo de subconsulta es particularmente útil cuando necesitas hacer referencia a datos externos o valores calculados sin depender de cada fila procesada en la consulta principal. Las subconsultas no correlacionadas pueden ser utilizadas en diferentes partes de una consulta, como en las cláusulas `SELECT`, `FROM`, `WHERE` o `HAVING`.

Sus principales características son:

1. **Son ejecutadas Independientemente**: se ejecuta una sola vez, de manera independiente, antes de que la consulta principal sea evaluada.
2. **No Depende de la Consulta Externa**: no requiere información de la consulta principal.
3. **Sencillez en la Escritura**: su estructura es más simple y puede reutilizarse en múltiples contextos

Las **subconsultas pueden devolver**:

- **Un solo valor** (realmente una tabla resultado de 1x1)
- **Un conjunto de valores** (una tabla de resultados)

Devuelve solo un valor: `WHERE`

```
SELECT
 E.nombre,
 E.apellido1,
 E.sueldo
FROM EMPLEADO E
WHERE Sueldo > ( SELECT AVG(Sueldo)
                 FROM EMPLEADO
                 WHERE Dpto = 5
                );
```

Tabla resultado:

nombre	apellido1	sueldo
Alberto	Campos	40000
Juana	Sáinz	43000
Fernando	Ojeda	38000
Eduardo	Ochoa	55000

- subconsulta en la cláusula `WHERE` y devuelve solo un valor
- el valor devuelto se usa como operando en la condición del `WHERE`
- en el ejemplo:
  - la subconsulta calcula y devuelve el importe promedio (33250) de los sueldos de los empleados del departamento 5
  - la consulta principal usa ese valor para compararlo con el sueldo de cada uno de los empleados, filtrando aquellos que no cumplan la condición

---

Devuelve solo un valor: `SELECT`

```
SELECT
 E.Nombre,
 E.Apellido1,
 E.Sueldo,
 ( SELECT AVG(Sueldo)
    FROM EMPLEADO
    WHERE Dpto = 5
  ) AS sueldoMedio5
FROM EMPLEADO E;
```

Tabla resultado:

nombre	apellido1	sueldo	sueldoMedio5
José	Pérez	30000	33250
Alberto	Campos	40000	33250
Alicia	Jiménez	25000	33250
Juana	Sáinz	43000	33250
Fernando	Ojeda	38000	33250
Aurora	Oliva	25000	33250
Luis	Pajares	25000	33250
Eduardo	Ochoa	55000	33250

- subconsulta en la cláusula `SELECT` y devuelve solo un valor
- el valor devuelto se usa como valor de una nueva columna
- en el ejemplo:

- la subconsulta calcula y devuelve el importe promedio de los sueldos de los empleados del departamento 5
- la consulta principal usa ese valor para mostrarlo en una nueva columna de la tabla resultado.

Devuelve solo un valor: `HAVING`

```
SELECT
 D.nombre,
 AVG(E.sueldo) AS sueldoMedio
FROM EMPLEADO E INNER JOIN DEPARTAMENTO D ON (E.dpto = D.numero)
GROUP BY D.numero, D.nombre
HAVING AVG(E.sueldo) > ( SELECT AVG(Sueldo)
                          FROM EMPLEADO
                          WHERE Dpto = 5
                        );
```

Tabla resultado:

nombre	sueldoMedio
Sede Central	55000

- subconsulta en la cláusula `HAVING` y devuelve solo un valor
- el valor devuelto se usa como operando en la condición del `HAVING`
- en el ejemplo:
  - la subconsulta calcula y devuelve el importe promedio de los sueldos de los empleados del departamento 5
  - la consulta principal usa ese valor para compararlo con el sueldo medio de cada uno de los departamentos, filtrando aquellos que no cumplan la condición

---

Ejercicio 02

Escribe una consulta que muestre los nombres de los departamentos cuyo sueldo medio de empleados sea superior al sueldo medio general de la empresa.

Solución:

Tabla resultado:

nombre	sueldoMedio
Sede Central	55000

Devuelve un conjunto de valores: `WHERE`

```
SELECT
 E.nombre,
 E.apellido1
FROM EMPLEADO E
WHERE Dni IN ( SELECT Empleado
                  FROM FAMILIAR
                  GROUP BY Empleado
                  HAVING COUNT(*) > 1
                 );
```

Tabla resultado:

nombre	apellido1
José	Pérez
Alberto	Campos

- subconsulta en la cláusula `WHERE` y devuelve un conjunto de valores
- el conjunto de valores devuelto se usa como operando en la condición del `WHERE`
- el operador `IN` comprueba si un valor específico aparece en un conjunto de resultados
- en el ejemplo:
  - la subconsulta calcula y devuelve el `Dni` de los empleados que tienen más de un familiar
  - la consulta principal usa ese valor para compararlo con el `Dni` de cada uno de los empleados, filtrando aquellos que no estén en la lista

---

Devuelve un conjunto de valores: `FROM`

```sql
SELECT
 D.nombre,
 S.SueldoMedio
FROM DEPARTAMENTO D
INNER JOIN ( SELECT dpto, AVG(Sueldo) AS sueldoMedio
              FROM EMPLEADO
              GROUP BY dpto
             ) AS S ON D.numero = S.dpto;
```

Tabla resultado:

nombre	sueldoMedio
Sede Central	55000
Administración	31000
Investigación	33250

- subconsulta en la cláusula `FROM` y devuelve un conjunto de valores
- el conjunto de valores devuelto se usa como una tabla intermedia en el `FROM`
- en el ejemplo:
  - la subconsulta calcula y devuelve el importe promedio de los sueldos de los empleados de cada departamento
  - la consulta principal usa ese valor para mostrarlo en una nueva columna de la tabla resultado junto al nombre de cada departamento

En algunos casos, una subconsulta no correlacionada se puede reescribir utilizando `JOIN`. Por ejemplo, la consulta anterior puede reescribirse como un `JOIN`:

```sql
SELECT
 D.nombre,
 AVG(Sueldo) AS sueldoMedio
FROM DEPARTAMENTO D
INNER JOIN EMPLEADO E ON D.numero = E.Dpto
GROUP BY Dpto;
```

Devuelve un conjunto de valores: `HAVING`

```sql
SELECT
 D.nombre,
```

```
 COUNT( distinct E.Dni) AS numEmpleados
FROM DEPARTAMENTO D INNER JOIN EMPLEADO E ON D.numero = E.Dpto
GROUP BY E.Dpto
HAVING COUNT( distinct E.Dni) NOT IN (
    SELECT
     COUNT( distinct empleado)
    FROM TRABAJA_EN
    GROUP BY proyecto);
```

Tabla resultado:

nombre	numEmpleados
Sede Central	1
Investigación	4

- subconsulta en la cláusula `HAVING` y devuelve un conjunto de valores
- el conjunto de valores devuelto se usa como como operando en la condición del `HAVING`
- `NOT IN` comprueba que un valor específico no aparezca en un conjunto de resultados
- en el ejemplo:
  - la subconsulta calcula y devuelve el número de empleados por proyecto
  - la consulta principal usa ese valor para compararlo con el número de empleados por departamento, filtrando aquellos que estén en el conjunto de resultados

---

Ejercicio 03

Escribe una consulta que liste el nombre y apellido de los empleados que trabajan en proyectos ubicados en 'Madrid'. Ordena los resultados alfabéticamente por apellido y nombre.

Solución:

```

```

Tabla resultado:

nombre	apellido1
Alberto	Campos
Eduardo	Ochoa

nombre	apellido1
Fernando	Ojeda
Juana	Sáinz

Subconsultas Correlacionadas

Subconsultas Correlacionadas

Las subconsultas correlacionadas **permiten resolver problemas más complejos y refinados**, ya que proporcionan la capacidad de hacer referencias cruzadas entre la consulta principal y la subconsulta. Pueden ser utilizadas en diferentes partes de una consulta, como en las cláusulas `SELECT`, `FROM`, `WHERE` o `HAVING`.

Sus características principales son:

1. **Dependencia de la Consulta Externa**: La subconsulta correlacionada hace referencia a columnas de la consulta principal. Su ejecución está ligada a los valores de cada fila de esa consulta.
2. **Ejecutada Varias Veces**: Se evalúa una vez por cada fila que procesa la consulta externa, lo que puede generar una mayor carga de procesamiento.
3. **Interacción entre Consultas**: A diferencia de las no correlacionadas, este tipo de subconsulta crea una interacción directa entre las dos consultas, ya que cada fila de la consulta principal afecta el resultado de la subconsulta.

Las **subconsultas pueden devolver**:

- **Un solo valor** (realmente una tabla resultado de 1x1)
- **Un conjunto de valores** (una tabla de resultados)

Devuelve solo un valor: `WHERE`

```
SELECT DISTINCT e.nombre, e.apellido1, e.sueldo
FROM EMPLEADO e
JOIN TRABAJA_EN t1 ON e.dni = t1.empleado
WHERE t1.horas > (
```

```
    SELECT MAX(t2.horas)
    FROM TRABAJA_EN t2
    JOIN EMPLEADO e2 ON t2.empleado = e2.dni
    WHERE e2.dpto = e.dpto    -- CORRELACIÓN
    AND t2.empleado != e.dni  -- CORRELACIÓN
    AND t2.horas IS NOT NULL
);
```

Tabla resultado:

nombre	apellido1	sueldo
Fernando	Ojeda	38000
Luis	Pajares	25000

- subconsulta en la cláusula `WHERE` y devuelve solo un valor
- el valor devuelto se usa como operando en la condición del `WHERE`
- la **subconsulta usa una columna de la consulta principal**
- en el ejemplo, encontrar empleados que trabajan más horas que cualquier otro empleado de su mismo departamento en cualquier proyecto:
  - la subconsulta calcula y devuelve el máximo de las horas trabajadas por un empleado del mismo departamento en cualquier proyecto para cada empleado (fila de consulta principal)
  - la consulta principal usa ese valor para compararlo con las horas de cada uno de los empleados, filtrando aquellos que no cumplan la condición

El **proceso de ejecución** de una subconsulta correlacionada sigue estos pasos:

1. La consulta principal selecciona una fila de la tabla.
2. La subconsulta se ejecuta usando los valores de esa fila.
3. La subconsulta devuelve un resultado que se utiliza para evaluar la condición en la consulta principal.
4. Este proceso se repite para cada fila de la consulta principal.

Devuelve solo un valor: `SELECT`

```
SELECT
 E1.nombre,
 E1.apellido1,
```

```
  E1.sueldo,
  ( SELECT AVG(E2.Sueldo)
      FROM EMPLEADO E2
      WHERE E1.Dpto = E2.Dpto  -- CORRELACIÓN
   ) AS sueldoMedio
FROM EMPLEADO E1;
```

Tabla resultado:

nombre	apellido1	sueldo	sueldoMedio
José	Pérez	30000	33250
Alberto	Campos	40000	33250
Alicia	Jiménez	25000	31000
Juana	Sáinz	43000	31000
Fernando	Ojeda	38000	33250
Aurora	Oliva	25000	33250
Luis	Pajares	25000	31000
Eduardo	Ochoa	55000	55000

- subconsulta en la cláusula `SELECT` y devuelve solo un valor
- el valor devuelto se usa como valor de una nueva columna
- la **subconsulta usa una columna de la consulta principal**
- en el ejemplo:
  - la subconsulta calcula y devuelve el importe promedio de los sueldos de los empleados del departamento que toca en cada ejecución (fila de consulta principal)
  - la consulta principal usa ese valor para mostrarlo en una nueva columna de la tabla resultado.

Devuelve solo un valor: `HAVING`

```
SELECT dpto, AVG(sueldo) as sueldoMedio
FROM EMPLEADO
GROUP BY dpto
HAVING AVG(sueldo) = (
    SELECT AVG(sueldo)
    FROM EMPLEADO e2
    WHERE e2.dpto = EMPLEADO.dpto  -- CORRELACIÓN
```

```
    AND e2.supervisor IS NOT NULL
);
```

Tabla resultado:

dpto	sueldoMedio
4	31000
5	33250

- subconsulta en la cláusula `HAVING` y devuelve solo un valor
- el valor devuelto se usa como operando en la condición del `HAVING`
- la **subconsulta usa una columna de la consulta principal**
- en el ejemplo:
  - la subconsulta calcula y devuelve el importe promedio de los sueldos de los empleados del departamento que toca en cada ejecución (fila de consulta principal) siempre que tengan supervisor
  - la consulta principal usa ese valor para compararlo con el sueldo medio de cada uno de los departamentos, filtrando aquellos que no cumplan la condición

---

Ejercicio 04

Escribe una consulta que liste el el nombre y apellido de cada empleado junto con la diferencia entre su sueldo y el sueldo medio de su departamento, ordena los resultados de mayor a menor diferencia de sueldo.

Solución:

```

```

Tabla resultado:

nombre	apellido1	diferenciaSueldoMedio
Juana	Sáinz	12000
Alberto	Campos	6750
Fernando	Ojeda	4750
Eduardo	Ochoa	0

nombre	apellido1	diferenciaSueldoMedio
José	Pérez	-3250
Alicia	Jiménez	-6000
Luis	Pajares	-6000
Aurora	Oliva	-8250

Devuelve un conjunto de valores: `WHERE`

```
SELECT
 nombre,
 apellido1
FROM EMPLEADO E
WHERE EXISTS (
    SELECT empleado
    FROM FAMILIAR F
    WHERE F.empleado = E.dni  -- CORRELACIÓN
    GROUP BY empleado
    HAVING COUNT(*) > 1
);
```

Tabla resultado:

nombre	apellido1
José	Pérez
Alberto	Campos

- subconsulta en la cláusula `WHERE` y devuelve un conjunto de valores
- el conjunto de valores devuelto se usa como operando en la condición del `WHERE`
- el operador `EXISTS` se evalúa verdadero si la consulta devuelve resultados
- en el ejemplo:
    - la subconsulta devuelve resultados si el empleado que toca en cada ejecución (fila de consulta principal) tiene más de un familiar
    - la consulta principal usa ese valor booleano para filtrar aquellos empleados que no tengan familiar

---

Devuelve un conjunto de valores: `HAVING`

```
SELECT
 D.nombre,
 AVG(E.sueldo) as sueldoMedio
FROM EMPLEADO E INNER JOIN DEPARTAMENTO D ON E.dpto = D.numero
GROUP BY dpto
HAVING EXISTS (
    SELECT empleado
    FROM TRABAJA_EN T
    JOIN EMPLEADO E2 ON T.empleado = E2.dni
    WHERE E2.dpto = E.dpto  -- ← CORRELACIÓN
    GROUP BY empleado
    HAVING COUNT(DISTINCT proyecto) > 1
);
```

Tabla resultado:

nombre	sueldoMedio
Administración	31000
Investigación	33250

- subconsulta en la cláusula `HAVING` y devuelve un conjunto de valores
- el conjunto de valores devuelto se usa como como operando en la condición del `HAVING`
- el operador `EXISTS` se evalúa verdadero si la consulta devuelve resultados
- en el ejemplo, encontrar departamentos donde existan empleados que trabajen en más de un proyecto:
    - la subconsulta devuelve los empleados del mismo departamento del empleado que toca en cada ejecución (fila de consulta principal) que trabajan en más de un proyecto
    - la consulta principal usa ese valor booleano para filtrar aquellos departamentos que no cumplen esa condición

---

Ejercicio 05

Escribe una consulta que liste el nombre de los proyectos donde trabaja al menos un empleado del departamento de 'Investigación', ordenado alfabéticamente.

Solución:

Tabla resultado:

nombre
Computación
ProductoX
ProductoY
ProductoZ
Reorganización

Uso de Subconsultas en DELETE y UPDATE

Uso de Subconsultas en DELETE y UPDATE

Además de su uso en `SELECT`, las subconsultas también se pueden utilizar en las sentencias `DELETE` y `UPDATE`. Esto nos permite eliminar o actualizar filas basándonos en los resultados de otra consulta, lo que es útil cuando las operaciones deben depender de datos relacionados o de condiciones que no se pueden expresar de manera sencilla sin una subconsulta.

Subconsultas en DELETE

```
DELETE FROM EMPLEADO
WHERE dni NOT IN (
 SELECT empleado
 FROM FAMILIAR
);
```

- se utilizan para eliminar filas de una tabla según condiciones dependientes de otra tabla o de otros datos.
- permiten identificar las filas que cumplen con ciertas condiciones basadas en una consulta adicional, antes de eliminarlas.
- en el ejemplo, se eliminan todos los empleados que no tienen famliares

Violación de integridad referencial

La consulta de este ejemplo no funcionará si tenemos activado el soporte de claves externas en SQLite porque viola la integridad referencial del atributo `director` en DEPARTAMENTO que es clave externa a `dni` en EMPLEADO.

Subconsultas en UPDATE: WHERE

```
UPDATE EMPLEADO
SET sueldo = sueldo + (sueldo*0.30)
WHERE dni IN (
 SELECT empleado
 FROM TRABAJA_EN
 GROUP BY empleado
 HAVING COUNT(DISTINCT proyecto) > 2
);
```

- se utilizan para actualizar filas de una tabla según condiciones dependientes de otra tabla o de otros datos.
- permiten identificar las filas que cumplen con ciertas condiciones basadas en una consulta adicional, antes de actualizarlas.
- en el ejemplo, se actualiza el sueldo a todos los empleados que trabajan en más de 2 proyectos.

Subconsultas en UPDATE: SET

```
UPDATE EMPLEADO
SET sueldo = (
 SELECT AVG(Sueldo)
 FROM EMPLEADO
)
WHERE dpto=5;
```

- se utilizan para actualizar filas de una tabla en base a un valor calculado mediante la subconsulta

- en el ejemplo, se actualiza el sueldo al sueldo medio total a todos los empleados que trabajan en el departamento 5.

Ejercicio 06

Aumenta un 20% el sueldo de los empleados que trabajan más horas totales que el promedio de horas trabajadas por los empleados que tienen horas trabajadas.

Después de la actualización, consulta la tabla empleado con:

```
SELECT nombre, apellido1, sueldo FROM EMPLEADO;
```

Solución:

```

```

Tabla resultado:

nombre	apellido1	sueldo
José	Pérez	36000
Alberto	Campos	48000
Alicia	Jiménez	30000
Juana	Sáinz	43000
Fernando	Ojeda	45600
Aurora	Oliva	30000
Luis	Pajares	30000
Eduardo	Ochoa	55000

Operadores típicos de subconsultas

Operador	Condición	Resultado	SQLite
IN	El valor de la consulta principal está en el conjunto devuelto por la subconsulta	`TRUE` si hay coincidencias	✅

Operador	Condición	Resultado	SQLite
NOT IN	El valor de la consulta principal **no** está en el conjunto devuelto por la subconsulta	`FALSE` si hay coincidencias	✅
EXISTS	La subconsulta devuelve al menos una fila	`TRUE` si la subconsulta devuelve resultados	✅
NOT EXISTS	La subconsulta **no** devuelve resultados	`FALSE` si la subconsulta devuelve resultados	✅
ALL	El valor de la consulta principal cumple la condición para **todos** los valores devueltos	`TRUE` si se cumple para todos	❌
ANY	El valor de la consulta principal cumple la condición para **al menos uno** de los valores	`TRUE` si se cumple para alguno	❌

- operadores fundamentales para trabajar con subconsultas, ya que permiten comparar un valor con un conjunto de resultados devueltos por una subconsulta de formas muy variadas
- `ALL` y `ANY`, aunque forman parte del estándar SQL, no están soportados en SQLite
- se puede conseguir el mismo comportamiento en SQLite haciendo que la subconsulta, en vez de devolver un conjunto de valores, devuelva solo un valor, el máximo o el mínimo de los valores. Las sustituciones serían:
  - `> ANY` → `> MIN`
  - `< ANY` → `< MAX`
  - `> ALL` → `> MAX`
  - `< ALL` → `< MIN`

EXISTS vs IN

EXISTS vs IN

`IN` y `EXISTS` permiten expresar la misma idea — *"quédate con las filas relacionadas con el resultado de la subconsulta"* — pero funcionan de forma distinta:

	`IN`	`EXISTS`
Qué compara	Un **valor** contra el **conjunto de valores** que devuelve la subconsulta	Comprueba si la subconsulta **devuelve alguna fila** (no importa qué columnas devuelva)
Tipo de subconsulta	Normalmente **no correlacionada** (se ejecuta una vez)	Normalmente **correlacionada** (se evalúa por cada fila de la consulta principal)
Evaluación	Materializa la lista completa y luego busca el valor	**Se detiene en la primera coincidencia** (cortocircuito): no necesita recorrer todas las filas
Rendimiento	Bien con listas **pequeñas**; puede degradarse con listas muy grandes	Suele rendir mejor con tablas **grandes** e **índices** sobre la columna de correlación
Negación con NULL	⚠️ `NOT IN` **falla silenciosamente** si la subconsulta devuelve algún NULL	✅ `NOT EXISTS` es **inmune a los NULL**

Optimizadores modernos

En consultas sencillas, los optimizadores actuales (incluido SQLite) suelen generar el mismo plan de ejecución para `IN` y `EXISTS`, así que la diferencia práctica de rendimiento puede ser nula. La diferencia que **siempre** se mantiene es la del comportamiento con NULL en la negación.

NOT IN vs NOT EXISTS: la trampa del NULL vuelve

En la lección de SELECT vimos que un `NOT IN` cuya lista contiene un `NULL` **nunca devuelve filas** (`x != NULL` es `NULL` y el `AND` nunca llega a ser verdadero). Con subconsultas, la trampa es mucho más peligrosa porque el NULL viene calculado de otra tabla y **no se ve**.

Consulta: empleados que no supervisan a nadie.

```sql
-- ✗ INCORRECTO: devuelve 0 filas
SELECT nombre, apellido1
FROM EMPLEADO
WHERE dni NOT IN (SELECT supervisor FROM EMPLEADO);
```

Tabla resultado: *(no hay resultados)*

¿Por qué? La subconsulta devuelve la columna `supervisor` de todos los empleados y Eduardo, que no tiene supervisor, aporta un **NULL** a la lista:

supervisor
333445555
888665555
987654321

Ese único NULL hace que la condición `NOT IN` nunca sea verdadera y la consulta devuelva una tabla vacía **sin dar ningún error**.

Hay dos soluciones. La primera es filtrar los NULL en la subconsulta:

```
-- ✓ CORRECTO con NOT IN: eliminando los NULL de la lista
SELECT nombre, apellido1
FROM EMPLEADO
WHERE dni NOT IN (SELECT supervisor
                  FROM EMPLEADO
                  WHERE supervisor IS NOT NULL);
```

La segunda, más robusta, es usar `NOT EXISTS`, que comprueba la existencia de filas y **no compara valores**, por lo que los NULL no le afectan:

```
-- ✓ CORRECTO y recomendado: NOT EXISTS
SELECT nombre, apellido1
FROM EMPLEADO E
WHERE NOT EXISTS (SELECT 1
                  FROM EMPLEADO S
                  WHERE S.supervisor = E.dni);  -- CORRELACIÓN
```

Tabla resultado (idéntica en ambos casos):

nombre	apellido1
José	Pérez
Alicia	Jiménez
Fernando	Ojeda
Aurora	Oliva
Luis	Pajares

Regla práctica

- Para la forma **afirmativa**, `IN` y `EXISTS` son intercambiables: elige el más legible.
- Para la forma **negativa**, usa `NOT EXISTS` por defecto. Si usas `NOT IN`, asegúrate **siempre** de que la subconsulta no pueda devolver NULL (columna `NOT NULL` o filtro `IS NOT NULL` explícito).
- Observa que este resultado coincide con el del **LEFT Anti-JOIN** de la lección de JOIN: son tres formas (anti-join, `NOT IN` saneado, `NOT EXISTS`) de expresar la misma consulta.

Ejercicio 07

Escribe una consulta que devuelva el `dni` de los empleados que no tienen ningún familiar registrado, ordenado por `dni`. Resuélvela con una de las siguiente opciones: (a) con `NOT IN` y (b) con `NOT EXISTS`. ¿Ha hecho falta sanear los NULL en la versión (a)? ¿Por qué?

Solución:

```

```

Tabla resultado:

dni
453453453
666884444
888665555
987987987
999887777

Subconsultas vs JOINs

Muchas consultas pueden definirse tanto con subconsultas como mediante JOINs, obteniéndose el mismo resultado. Sin embargo, es fundamental tener en cuenta el impacto en

el rendimiento y evaluar si una subconsulta es la opción más eficiente o si se puede lograr el mismo resultado mediante un `JOIN`. En general, podemos decir:

- **Subconsultas**: Se ejecutan de forma separada de la consulta principal, lo que puede ser útil cuando solo se necesita un valor o conjunto de valores específico.
- **JOINs**: Combinan tablas directamente, lo que puede ser más eficiente para grandes conjuntos de datos, pero puede no ser adecuado si solo necesitas un valor calculado específico, como el resultado de un `AVG`, `SUM`, o `COUNT`

Subconsultas

Ventajas	Inconvenientes
Legibilidad en lógica compleja: Para consultas con múltiples niveles de filtrado, pueden ser más intuitivas de leer	**Rendimiento**: Generalmente más lentas, especialmente si se ejecutan por cada fila (subconsultas correlacionadas)
Modularidad: Permiten resolver el problema paso a paso, facilitando el desarrollo y debugging	**Menor optimización**: Los optimizadores de consultas suelen tener más dificultad para optimizarlas
Útiles para comparaciones agregadas: Excelentes cuando necesitas comparar un valor con un agregado (MAX, AVG, etc.)	**Dificultad de mantenimiento**: Las subconsultas anidadas pueden volverse difíciles de seguir
No generan duplicados: En subconsultas escalares o con EXISTS, no hay riesgo de multiplicación de filas	

JOINs

Ventajas	Inconvenientes
Rendimiento superior: Más eficientes, especialmente con índices adecuados	**Complejidad visual**: Con muchas tablas, la sintaxis puede volverse difícil de leer
Mejor optimización: Los motores de bases de datos están altamente optimizados para JOINs	**Riesgo de duplicados**: Un JOIN incorrecto puede multiplicar filas inesperadamente
Flexibilidad: Permiten combinar múltiples columnas de diferentes tablas fácilmente	**Curva de aprendizaje**: Los diferentes tipos de JOIN (INNER, LEFT, RIGHT, FULL) requieren comprensión clara
Estándar para relaciones: Son la forma natural de trabajar con bases de datos relacionales	

Ejercicio 08

Reescribe la siguiente consulta que utiliza JOINs para que use **subconsultas** en su lugar. Ordena los resultados por apellido y nombre.

```sql
-- Empleados que trabajan en proyectos ubicados en 'Madrid'
SELECT DISTINCT
  E.nombre,
  E.apellido1
FROM EMPLEADO E
INNER JOIN TRABAJA_EN T ON E.dni = T.empleado
INNER JOIN PROYECTO P ON T.proyecto = P.numero
WHERE P.ubicacion = 'Madrid';
```

Solución:

```sql

```

Tabla resultado:

nombre	apellido1
Alberto	Campos
Eduardo	Ochoa
Fernando	Ojeda
Juana	Sáinz